

Snake & Ladder Game — Final Project Report

Course: CMPSC 132 | Name: Faiz Daroga

Project Description

For my CMPSC 132 final project, I built a terminal-based Snake & Ladder game in Python. It supports two human players or one player against the computer. Players take turns rolling a six-sided die and moving along a board numbered 1 to 100 — land on a snake's head and you slide down, land at the base of a ladder and you climb up. First to hit exactly cell 100 wins. If a roll would push you past 100, you stay put.

Beyond the core requirements, I added three extra credit features: a custom Stack class for move history, three difficulty levels with different board configurations, and a persistent win stats system that saves results to a JSON file between sessions.

Approach and Logic

I kept things organized by giving each function one clear job. `main()` shows the leaderboard and repeatedly calls `play_game()` until the user quits. Inside `play_game()`, the program collects a difficulty level, game mode, and player names before entering the main loop. Each iteration calls `print_board()` to show the current state, then `take_turn()` for each player — which handles everything from input and dice rolling to position updates, overshoot checks, snake/ladder lookups, and logging the move. An `is_computer` parameter (default `False`) lets the same function handle both player types cleanly, adding a `time.sleep(1.2)` pause for the computer so the human can follow along. Input validation runs throughout using `while True` loops, with blank names falling back to defaults and a `q` option available before any roll.

Data Structures

Dictionaries are the backbone of the game. Snakes and ladders are each stored as a dictionary mapping a start cell to a destination — for example, `snakes = {16: 6, 99: 5}`. This keeps lookups $O(1)$ instead of looping through a list every turn. Three separate pairs of these dictionaries handle the easy, medium, and hard difficulty settings.

I also built a custom Stack class to record move history, backed by a Python list but exposing proper methods like `push()`, `pop()`, `peek()`, and `get_last()`. It models the LIFO principle directly

and ties into the abstract data types we covered in class. At the end of each game, `get_last(10)` pulls the ten most recent moves for a recap.

Win stats are saved to `stats.json` as a dictionary of player names mapped to win counts. It loads at startup and writes back after each win, with a `try/except` block so a corrupted file never crashes the game.

Challenges and Improvements

The trickiest part was getting the board to render correctly. A real Snake & Ladder board alternates direction each row — left to right, then right to left, then left to right again. I had to use the row index to pick direction dynamically, and getting the ranges right took a few attempts. Adding difficulty levels also caused some refactoring: snakes and ladders started as global variables, but supporting three configurations meant converting them into parameters passed through every function call.

If I were to take this further, the most impactful addition would be a Tkinter GUI with a visual board. I'd also refactor the code into `Player`, `Board`, and `Game` classes to make it cleaner and easier to extend — and eventually support more than two players by switching from hardcoded variables to a list-based player system.